SMART FILE & MEDIA MANAGER

Advanced Programming | Assessment 2 | Technical Report and Evaluation

Individual desktop application project — Python and PyQt6

Report status: source code, automated model/scanner tests and a Linux-based metadata benchmark have been created and checked. PyQt6 GUI execution, Windows deployment, manual GUI evidence, real remote Git activity and a live in-class demonstration still require completion on the student's computer. This report distinguishes confirmed results from planned checks.

Project	Summary
Theme	Local file and media processing manager
Platform	Windows 10/11 target; PyQt6 desktop GUI
Key techniques	QMainWindow; MVC-style separation; QThread; Qt signals; QtMultimedia; Python unittest; Git; PyInstaller build script
Assessment brief	Assessment 2: 15 tasks; 8 official deliverables; 70 available marks

1. Purpose, user and requirements

The application is designed for someone who wants to inspect and organise information about many local files without opening them one at a time. They can choose a directory, scan files, see metadata, filter results, preview supported media and export a CSV inventory. Scans read the source files without changing or uploading them.

ID	Functional requirement
FR01	Choose an existing local folder
FR02	Scan files with optional subfolders
FR03	Display sortable metadata table
FR04	Search and filter results
FR05	Preview common images and limited text
FR06	Preview audio with QtMultimedia
FR07	Show progress and cancel background processing
FR08	Export filtered data to CSV
FR09	Handle invalid paths, unreadable files and export failures

2. GUI plan and widget choices

The QMainWindow includes File/Help menus, a folder field and Browse action, recursive scan checkbox, Scan and Cancel buttons, search field, category selector, sortable QTableWidget, a preview area, progress bar and status bar. A QSplitter lets users allocate space between the results table and the preview. The controls support the main workflow rather than adding widgets simply for variety.

The initial wireframe is reproduced in docs/DESIGN.md. The table is the main navigation surface; preview selection shows relevant file details. Errors use a message dialog while ordinary progress uses the status bar.

3. Architecture and separation of concerns

Part	Responsibility / file
Model	app/models.py: immutable FileRecord data, categories, size/date formatting and filtering. app/scanner.py: folder discovery, metadata extraction and limited text preview.
View	app/view.py: QMainWindow, menus, layouts, table, image/text/audio preview, progress and user dialogs.
Controller	app/controller.py: connects GUI signals, starts/cancels workers, applies filters, exports CSV and coordinates updates.
Worker	app/workers.py: QObject moved to QThread; filesystem work performed away from GUI, results emitted via Qt signals.

Interaction sequence: user clicks Scan → controller validates the folder → QThread starts worker → worker discovers files and extracts metadata → signals report progress and results → controller refreshes the view.

GUI widget manipulation is intentionally limited to the main thread.

4. Events, validation and exception handling

Browse, scan, cancel, table selection, search, category selection, audio playback and CSV export are connected through Qt signals. The controller checks that the target folder exists before scanning. The scanner catches individual OSError failures and continues processing other files; errors are reported to the user rather than crashing the whole scan. CSV output catches write failures. Unknown preview types display an explanatory message.

5. Multimedia

Audio previews use QMediaPlayer with QAudioOutput from QtMultimedia, including Play, Pause and Stop. A short bundled WAV notification is played by QSoundEffect when a scan finishes. This directly helps users know when long work has completed; audio codec availability is operating-system dependent.

6. Concurrency and asynchronous processing

Folder discovery and metadata collection can take time for large directories. Executing this directly inside a button callback would freeze the event loop. The application therefore moves ScanWorker onto a QThread. The worker emits progress, status, completed and finished signals. During file discovery the progress bar is indeterminate; after the file count is known, progress becomes determinate. The view remains available for interaction, and the Cancel button requests a safe stop.

7. Synchronisation and thread safety

Shared state includes scan cancellation status and the final file list displayed by the GUI. A threading.Event allows the GUI to set a cancellation request safely while the worker checks it regularly. The worker does not change any widgets directly. It delivers the completed list through a signal; the controller owns and updates the displayed records on the GUI thread. Without these rules, operations could read inconsistent state or touch widgets from a non-GUI thread, producing unpredictable behaviour. Closing while a scan is active asks it to stop before allowing the window to close.

8. Performance and parallelism investigation

A repeatable benchmark compares sequential metadata inspection against Python ThreadPoolExecutor.map with four workers, using 800 generated small local files and three runs. The inspected metadata and total byte count are equal for both methods. Times are measured with time.perf_counter(). The following actual readings were obtained while preparing this package on Linux; they are not claimed as measurements on the student's Windows device.

Run	Sequential (s)	Threaded (s)
1	0.005571	0.033172
2	0.007782	0.082996
3	0.006739	0.029109
Average	0.006697	0.048426

Measured speed-up is sequential/threaded = 0.14x. In this local test, using a thread pool was slower: each stat call is very short, and thread scheduling, task submission and result collection cost more than the work saved. This is evidence that parallelism is not automatically faster.

I/O-bound versus CPU-bound work

Reading file attributes is mainly an I/O/system-call activity; threads can help when operations spend time waiting, especially on slow or remote drives. With small cached local files, the scheduling overhead dominates. CPU-heavy Python loops behave differently: the CPython GIL often restricts CPU-bound Python threads, so multiprocessing or native code may be more effective. Scalability depends on workload size, disk speed, number of workers and contention. A parallel query can be represented using executor.map to apply the same operation to many file items and combine its results; a .NET PLINQ dependency is not required for this Python assignment.

Amdahl's Law

S(N) = 1 / ((1 − P) + P / N). If the parallelisable fraction P = 0.85 and four processors are available, S(4) = 1 / (0.15 + 0.85/4) = **2.76×**. The serial 15% sets the limit even under ideal conditions. These P and N values are a hypothetical illustration, not parameters measured in the benchmark.

How to repeat the investigation

Run: python benchmarks/benchmark.py --demo --repeats 3 --workers 4 --output evidence/own_benchmark.csv. Repeat with an actual large local directory and describe differences, rather than presenting the package-building environment results as personal measurements.

9. Test plan

The test plan in docs/TEST_PLAN.md maps requirements to tests, includes normal conditions, empty directories and invalid or exceptional inputs, and provides fields for actual outcomes and screenshots. It explicitly separates unperformed manual GUI tests from automated checks.

10. Manual testing

Required manual checks include navigation, file selection and preview, empty folders, invalid folder paths, inaccessible files, audio playback, export errors, large scans, cancellation/restart and closing during scanning. These tests remain PENDING in this package because the PyQt6 GUI and Windows target could not be executed in the assembly environment. Results must be recorded on the target machine.

11. Automated testing

Thirty Python unittest cases were executed successfully (30 passed; no failures) against the Qt-independent data/scanning/benchmark logic. Tests cover extension categorisation, formatting boundaries, case-insensitive filters, formula-safe CSV cells, recursive discovery, hidden files, cancellation, missing files, text preview limits and agreement between sequential and threaded inspections. The original test log is stored in evidence/automated_tests.txt.

12. Debugging and corrective development

The provided test plan has a defect/correction table with columns for the problem, cause, fix, Git commit and rerun results. No unverified GUI bugs or invented fixes are presented as historical facts. The student should record genuine defects found while testing on Windows, correct them and include evidence of rerunning the affected tests.

13. Git version control

A local Git history is included to illustrate tracked project development with logical commits and branch merges. This packaging history must not be misrepresented as the student’s own long-term development. The student should make further genuine commits, show meaningful branches and merges, and document a conflict if one occurs or through a controlled exercise. Remote push/pull/review should be demonstrated when institutional arrangements allow; see docs/GIT_WORKFLOW.md.

14. Distribution

The package provides dependency lists and run_windows.bat to create a Python virtual environment, install PyQt6 and start main.py. build_windows.bat installs PyInstaller and builds a Windows onedir distribution with the sound asset. This is a build configuration, not a pre-tested Windows executable. Packaging and execution must be verified on Windows with screenshots; see docs/BUILD_AND_VERIFICATION.md.

15. Evaluation and presentation

Strengths include clear project separation, a responsive asynchronous scan, safe cancellation, useful metadata filters, both media preview and notification, CSV export and reproducible tests. Limitations include memory use for a large QTableWidget, OS-dependent audio codecs, a basic image preview rather than generated thumbnails, lack of persistent indexing, and a separate performance CLI instead of a built-in analysis view. Future improvements could include paged models, thumbnail caching, an SQLite index and a configurable scan error log. Presentation slides are supplied in presentation/Project_Presentation.pptx; an in-class live demonstration remains the student’s responsibility.

Final submission checklist

Deliverable	Location / status
Python/PyQt source	main.py + app/ — assembled; GUI must be run on Windows
Git history	.git + docs/GIT_WORKFLOW.md — example local history; student activity required
Automated tests	tests/ + evidence/automated_tests.txt — 30 passed
Test plan/results	docs/TEST_PLAN.md — manual actual results pending
Technical documentation	docs/Technical_Report.pdf and docs/DESIGN.md
Performance study	benchmarks/benchmark.py + evidence/benchmark_results.csv/.txt — Linux demo measured
Build/distribution	build_windows.bat + requirements-build.txt — Windows packaging pending
README	README.md — setup and run instructions
Additional presentation	presentation/Project_Presentation.pptx — live demonstration pending

References (original documentation)

Qt Documentation. QMainWindow. https://doc.qt.io/qt-6/qmainwindow.html

Qt Documentation. QThread. https://doc.qt.io/qt-6/qthread.html

Qt Documentation. QMediaPlayer. https://doc.qt.io/qt-6/qmediaplayer.html

Qt Documentation. QSoundEffect. https://doc.qt.io/qt-6/qsoundeffect.html

Python Software Foundation. concurrent.futures. https://docs.python.org/3/library/concurrent.futures.html

Python Software Foundation. unittest. https://docs.python.org/3/library/unittest.html

Python Software Foundation. time.perf_counter. https://docs.python.org/3/library/time.html#time.perf_counter

PyInstaller Documentation. https://pyinstaller.org/en/stable/